Лабораторная работа №2
Распределенные приложения на Java с использование API java.net

Цель работы

Целью работы является ознакомление с основными принципами реализации сетевого взаимодействия с использованием одного из двух транспортных протоколов: UDP и TCP.

1. Теоретический материал

Понятие сетевого взаимодействия

Рассмотрим вопросы разработки распределенных приложений, передающих данные по сети. Такие приложения называют еще сетевыми.

Сетевые программы выполняются на нескольких устройствах (компьютерах), которые подключены друг к другу с использованием сети. Компьютеры, работающие в Интернете, обмениваются данными друг с другом, используя либо Протокол управления передачей (*Transmission Control Protocol* - TCP), либо Протокол пользовательских дейтаграмм (*User Datagram Protocol* - UDP). Сетевые программы предназначены для передачи файлов, взаимодействия с базой данных или электронной почты.

Процесс передачи данных по сети от одного приложения к другому выполняется на нескольких уровнях (рис.1):

физическом;

сетевом;

транспортном;

прикладном.

На физическом уровне задаются характеристики передающей среды, скорость передачи данных и т.д.

Сетевой уровень связан с обменом данными между некоторым узлом (рабочей станцией, сервером и т.д.) и сетью, к которой подсоединен этот узел. Узел-отправитель должен задать адрес узла-адресата, которому должны быть отправлены данные. На этом уровне осуществляется маршрутизация, т.е. переход между отдельными сетями, соединяющими взаимодействующие узлы. Маршрутизация выполняется с использованием Internet-протокола (IP).

Независимо от природы приложений обмен данными должен быть надежным, т.е. необходимо гарантировать доставку данных приложению-адресату и последовательность отдельных порций данных (пакетов) в том порядке, в котором они были отправлены. Механизмы обеспечения надежности независимы от приложений и выделены в один общий уровень - транспортный. Как правило, на этом уровне применяется протокол управления передачей TCP и реже UDP.

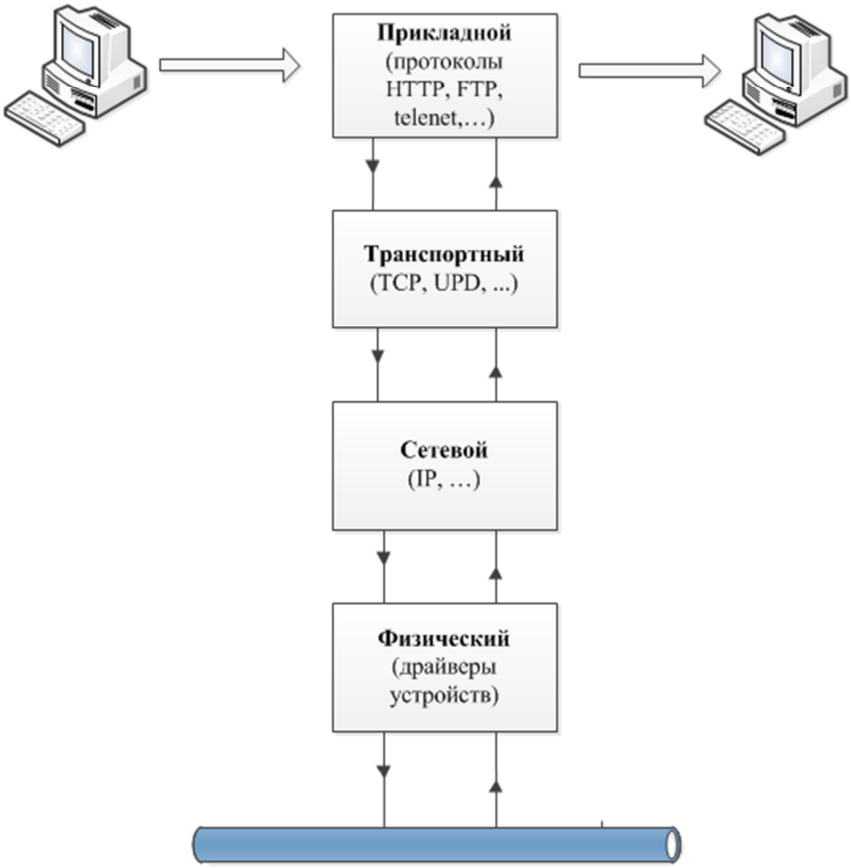

Рис. 1 Передача данных по сети

На прикладном уровне (уровне работы приложений) заложена прикладная логика, необходимая для обеспечения работы пользовательских приложений. Именно разработке таких пользовательских приложений посвящена данная работа.

Протоколы транспортного уровня

Передаваемые по сети данные, во-первых, должны быть направлены узлу, на котором находится приложение-адресат, и, во-вторых, переданы на этом узле приложению, которому они предназначены. Причем на узле одновременно могут выполняться несколько различных приложений.

Таким образом, нужно указать два уровня адресации:

- уникальный глобальный сетевой адрес узла, на котором находится приложения адресат. Данный адрес представляет собой 32-битный адрес для протокола IPv4 или 128-битный для IPv6. Часто его представляют в символьной форме mmm.nnn.ppp.qqq (четыре однобайтовых поля в виде 4-х десятичных чисел, разделённых точками).

- работающее на узле приложение должно иметь уникальный адрес в пределах этого узла, позволяющий транспортному протоколу доставить данные нужному приложению. Этот внутренний адрес приложения в терминологии протоколов сетевого взаимодействия называют портом. Номер порта изменяется в диапазоне от 0 до 65535 (для протокола TCP).

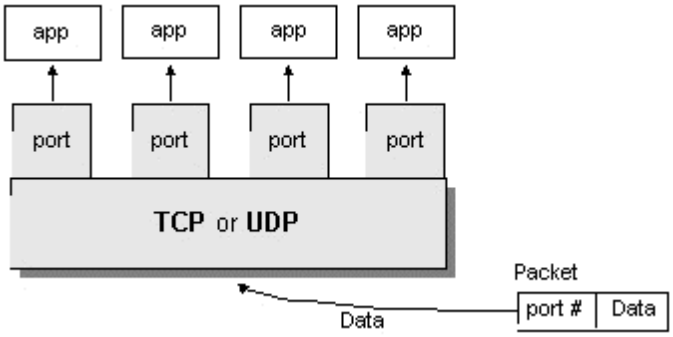

Рис.2 Использование портов

Эта пара адресов, представляющая конечную точку соединения, называется ***сокетом***, т.е.«гнездом», соответствующим сетевому адресу и порту.

На транспортном уровне используется 2 протокола: UDP и TCP.

Пользовательский протокол датаграмм UDP (User Datagram Protocol) предоставляет сервис без установления соединения и не гарантирует доставку, сохранение последовательности переданных пакетов, защиту от их дублирования.

Для большинства приложений протоколом транспортного уровня выступает TCP (Transmission Control Protocol). Этот протокол обеспечивает надежное соединение для передачи данных от одного приложения к другому.

Работа с этими протоколами в Java используются классы пакета java.net. Java-программы, которые обмениваются данными по сети, работают на уровне приложения. Как правило, разработчику не нужно беспокоиться о слоях передачи данных, необходимо лишь воспользоваться классами в пакете java.net. Эти классы обеспечивают системно-независимую сетевую связь. Однако, чтобы решить, какие классы Java должны использовать ваши программы, нужно понять, чем отличаются TCP и UDP.

Использование протокола UDP API java.net

Протокол UDP обеспечивает связь, которая не гарантируется между двумя приложениями в сети. UDP не использует установленных соединений. Скорее, он отправляет независимые пакеты данных, называемые дейтаграммами, из одного приложения в другое. Отправка дейтаграмм очень похожа на отправку письма через почтовую службу: порядок доставки не важен и не гарантируется, и каждое сообщение не зависит от другого.

Рассмотрим, например, сервер синхронизации, который отправляет текущее время своему клиенту, когда этого требует. Если клиент пропускает пакет, серверу не имеет смысла выполнять повторную посылку, потому что время будет неверным, когда клиент получит его во второй попытке. Если клиент делает два запроса и принимает пакеты с сервера не по порядку, это не имеет особого значения, поскольку клиент может выяснить, что пакеты не соответствуют порядку и делают другой запрос. В этом случае надежность

TCP не нужна, поскольку это приводит к ухудшению производительности и может помешать полезности службы.

Другим примером службы, которая не нуждается в гарантии надежного канала, является команда ping. Цель команды ping - проверить связь между двумя программами по сети. Фактически, ping должен знать о потерянных или нестандартных пакетах, чтобы определить, насколько хорошо или плохо установлена связь. Надежный канал (посылка с подтверждением) полностью обесценивает эту услугу.

UDP используется для передачи пакетов данных (дейтаграмм). Длина одной дейтаграммы этого протокола ограничена 65536 байтами, поэтому для пересылки данных большего размера они должны быть разбиты на отдельные пакеты отправителем и снова собраны получателем. Передача сообщения - не блокирующая, прием - блокирующий с возможностью прерывания по истечению времени ожидания.

Для работы с UDP в пакете java.net используются следующие классы:

- **`DatagramPacket`** (дейтаграмма) - объект класса представляет единичную дейтаграмму (сообщение). В конструктор этого класса передается массив байт и адрес процесса-получателя (IP адрес узла и порт). Этот класс используется для создания сообщения как перед его отправкой, так и при приеме (функция приема возвращает экземпляр этого класса)
- **`DatagramSocket`** **–** объект класса предназначен для посылки/приема UDP дейтаграмм. В конструктор класса передается порт, с которым связывается сокет. Другой конструктор, без аргументов, использует в качестве порта первый свободный порт. Класс имеет методы `send` для передачи и `receive` для приема дейтаграмм. Метод `setSoTimeout` устанавливает таймаут для операций сокета.

Ниже приведены простые программы, использующие рассмотренные механизмы для организации взаимодействия.

Упражнение 1. Программа клиента создает сокет, соединяется с сервером (порт 6789), пересылает ему сообщение и ждет ответа.

Программа сервера создает сокет и обслуживает запросы клиента.

1) Класс клиента UDPClient

```
public class UDPClient{
  public static void main(String args[]){
    // аргументы args – сообщение и адрес сервера
    try {
    // создание сокета
    DatagramSocket aSocket = new DatagramSocket();
    //  Сообщение
    byte [] message = args[0].getBytes();
```

```
      //  Адрес и порт сервера
      InetAddress aHost = InetAddress.getByName(args[1]);
      int serverPort = 6789;
      //  Создание пакета
      DatagramPacket request =new DatagramPacket
            (message,args[0].length(),aHost, serverPort);
      //  Отправка сообщения
      aSocket.send(request);
      //  Создание буфера для приема ответа сервера
      byte[] buffer = new byte[1000];
      DatagramPacket reply = new DatagramPacket(
             buffer, buffer.length);
      //  Ожидание ответа
      aSocket.receive(reply);
      System.out.println("Reply: "
             + new String(reply.getData()));
      //  Закрытие сокета
      aSocket.close();
      } catch (SocketException e){
        //  Ошибка создания сокета
        System.out.println("Socket: " + e.getMessage());
      } catch (IOException e){
        //  Ошибка при приеме
        System.out.println("IO: " + e.getMessage());
      }
    }
  }
```

Пояснения

Адрес сервера и сообщение задаются через аргументы запуска приложения. Порт не передается, т.к. известен заранее. При создании датаграммы в ее конструктор передается массив с передаваемыми данными, адрес узла, на котором выполняется сервер и порт. Пакет передается на сервер. Адрес получателя определяется в пакете, а не в сокете, через который мы его передаем. Т.е. один и тот же сокет может быть использован для передачи пакетов разным узлам. И этот же сокет используется для приема пакета от сервера.

2) Класс сервера UDPServer

```
public class UDPServer{
  public static void main(String args[]) {
```

```
      DatagramSocket aSocket = null;
      try{
        //  Создание сокета на заданном порту
        aSocket = new DatagramSocket(6789);
        //  Массив байт для приема данных
        byte[] buffer = new byte[1000];
        while(true){
          //  Создание датаграммы
          DatagramPacket request =
               new DatagramPacket(buffer, buffer.length);
          // Прием пакета данных
          aSocket.receive(request);
          //  Формирование пакета ответа – копия принятого
          DatagramPacket reply = new DatagramPacket(
               request.getData(), request.getLength(),
               request.getAddress(), request.getPort());
          //  Отправка ответа
          aSocket.send(reply);
        }
      } catch (SocketException e){
        //  Ошибка создания сокета
        System.out.println("Socket: " + e.getMessage());
      } catch (IOException e) {
        //  Ошибка приема- отправки данных
        System.out.println("IO: " + e.getMessage());
      } finally {
        //  Закрытие сокета
         if(aSocket != null) aSocket.close();
      }
   }
}
```

Выполнение приложения

Первым запускается сервер, который после запуска переходит в режим ожидания сообщений от клиента. Поскольку нами не предусмотрено никакого механизма останова сервера, выгрузить его может либо произошедшая ошибка, либо насильственное прерывание процесса.

Клиент запускается с заданием параметров: первый – сообщение (например, TestUDP), второй – адрес сервера (это локальный адрес localhost или 127.0.0.1, т.к. и сервер и клиент запускаются на локальном компьютере). Строка параметров: `ТестUDP 127.0.0.1`.

Параметры задаются в редакторе конфигурации клиентского приложения. Для этого выделив класс клиента необходимо выполнить `Run – Run Configuration` и далее на вкладке `Arguments` задать параметры, разделив их пробелом.

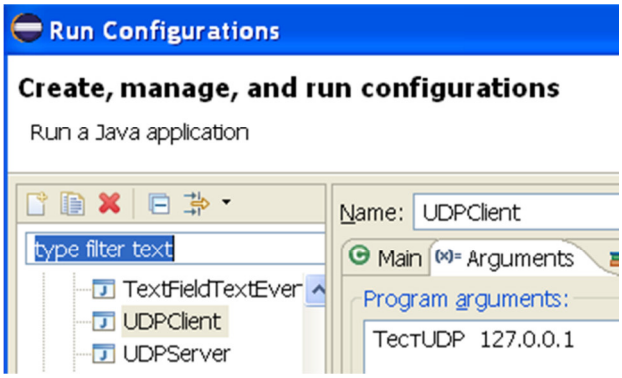

Рис.3 Задание параметров запуска клиента

После запуска клиент отправляет строку на сервер, выводит в консоль ответ сервера и завершает работу (Рис. 4).

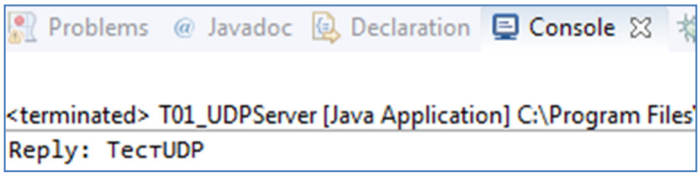

Рис. 4 Результат работы клиента

Использование протокола TCP API java.net

TCP - это протокол на основе соединений, обеспечивающий надежный поток данных между двумя компьютерами.

Когда два приложения хотят надежно связываться друг с другом, они устанавливают соединение и отправляют данные по этому соединению. Это аналогично телефонному звонку. Если вы хотите поговорить с абонентом, то предварительно устанавливается связь, когда вы набираете его номер телефона, и абонент отвечает. Далее абоненты разговаривают, отправляя данные по телефонному каналу. Как и телефонная компания, TCP гарантирует, что данные, отправленные с одного конца соединения, фактически попадают на другой конец и в том же порядке, который был отправлен. В противном случае сообщается об ошибке.

TCP обеспечивает канал «точка-точка» для приложений, требующих надежной связи. Протокол передачи гипертекста (HTTP), протокол передачи файлов (FTP) и Telnet - все это примеры приложений, требующих надежного канала связи. Порядок, в котором данные отправляются и принимаются по сети, имеет решающее значение для успеха этих приложений. Когда HTTP используется для чтения с заданного URL-адреса, данные должны быть получены в том порядке, в котором он был отправлен. В противном случае вы получите перемешанный HTML-файл, поврежденный zip-файл или

другую недопустимую информацию.

Протокол использует потоки при передаче-приеме данных. Для передачи данных создается TCP сокет, с которым связано два потока - поток ввода и поток вывода. Далее сокет использует связанный с ним поток и пишет данные в поток или читает данные из потока. В отличие от протокола UDP данный протокол не накладывается ограничений на размер сообщения. По истечении таймаута если не получено от получателя подтверждение приема данных, то выполняется повторная передача данных.

Протокол использует два класса:

`ServerSocket` для прослушивания запросов клиента на стороне сервера

`Socket` для работы с соединением как на стороне клиента, так и сервера.

Класс `ServerSocket` используется приложением сервера для получения порта и прослушивания запросов клиента. Для сокета задаются (в конструкторе или через его методы): порт, количество входящих клиентов, локальный IP-адрес для приема клиентских запросов (используется для серверов, которые могут иметь несколько IP-адресов). Экземпляр сокета используя метод `accept()` ожидает входящего клиента. Этот метод блокирует сокет до тех пор, пока клиент не подключится к серверу по указанному порту или не истечет тайм-аут сокета (если тайм-аут установлен методом `setSoTimeout`).

Класс `Socket` представляет собой сокет, используемый как клиентом, так и сервером для связи друг с другом. Клиент непосредственно создает объект `Socket`, а сервер получает объект `Socket` из возвращаемого значения метода `accept()` класса `ServerSocket`. Для соединения с удаленным узлом используется метод `connect()`. Методами `getInputStream()` и `getOutputStream()` получаются входной и выходной потоки (соответственно для приема и передачи данных).

Вспомогательный класс `InetAddress` определяет интернет IP-адрес.

Примеры приложений использующих сокеты

Ниже приведены простые программы иллюстрирующие использование рассмотренных классов.

Упражнение 2. Однонаправленная связь. Только сервер передает сообщение. По запросу клиента сервер возвращает текущее время.

Класс сервера `TimeServer`.

TCP сервер запущен на заданном порту 9090. После подсоединения клиента сервер посылает клиенту текущую дату и время, а затем закрывает соединение клиента

```
public class TimeServer {
   public static void main(String[] args) throws IOException {
```

```
      //  Создание сокета для прослушивания клиентов
      ServerSocket listener = new ServerSocket(9090);
      System.out.println("Сервер ожидает запрос клиента ...");
      try {
         //  Цикл ожидания запроса клиента
         while (true) {
            Socket socket = listener.accept();
            try {
               // Извлечение выходного потока из сокета
              PrintWriter out=new PrintWriter(
                     socket.getOutputStream(),true);
                // Передача ответа клиенту
                out.println(new Date().toString());
            } finally {
                socket.close();
            }
         }
      } finally {
        listener.close();
      }
   }
}
```

Класс клиента **TimeClient**

Клиент при запуске в окне диалога запрашивает адрес сервера (IP-адрес), после подсоединяется к серверу по сокету и отображает дату полученную от сервера.

```
public class TimeClient {
   public static void main(String[] args)
             throws IOException{
      //  Вывод диалога для ввода адреса сервера
      String serverAddress = JOptionPane.showInputDialog
           ("Введите IP-адрес сервера времени\n" +
              "на порту 9090:");
      Socket s = null;
      try {
        //  Создание сокета для заданного адреса сервера
        s = new Socket(serverAddress, 9090);
        //  Создание входного потока
        BufferedReader input = new BufferedReader(
           new InputStreamReader(s.getInputStream()));
        //  Чтение ответа
        String answer = input.readLine();
        //  Вывод времени в окне диалога
        JOptionPane.showMessageDialog(null, answer);
```

```
        //  Завершение работы клиента
      } finally {
         c.close();
      }
      System.exit(0);
    }
}
```

Запуск приложения

Вначале запускаем сервер. В консоли будет выведено сообщение о готовности сервера к работе (рис. 5).

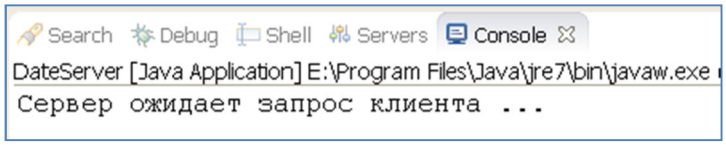

Рис. 5 Запуск сервера

Затем запускаем клиента. Клиент выводит окно для ввода адреса сервера (рис.6). После ввода адреса (localhost или 127.0.0.1) клиент устанавливает соединение с сервером, получает ответ от сервера в виде текущего времени, выводит его в диалоговом окне (рис. 7) и завершает работу.

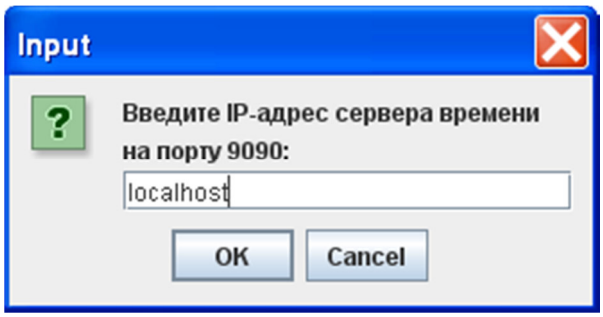

Рис. 6 Запуск клиента и запрос адреса

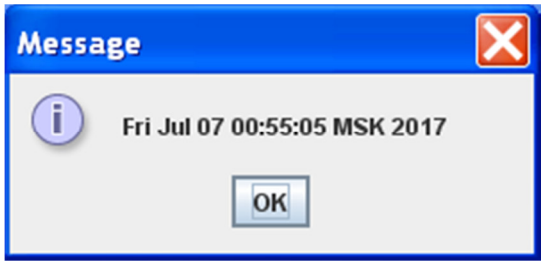

Рис. 7 Получение ответа от сервера

Следующий пример иллюстрирует двунаправленную связь, сервер обслуживает нескольких клиентов.

Упражнение 3. Обеспечение двунаправленного обмена. Клиент передает на сервер текстовую строку. Сервер принимает запросы от клиентов, преобразует строку в верхний регистр и возвращает клиенту.

Класс сервера `ConvStrServer`

Когда новый клиент подключаются, сервер создает новый поток сетевого взаимодействия, в котором клиент отправляет строку, а сервер отправляет обратно строку преобразованную в верхний регистр. Программа сервера запускается в бесконечном цикле, поэтому ее снятие выполняется через прерывание - завершения приложения.

```
public class ConvStrServer {
   public static void main(String[] args) throws Exception {
     System.out.println("Сервер преобразования запущен");
     int clientNumber = 0;
     //  Создание сокета для прослушивания клиентов
     ServerSocket listener = new ServerSocket(9898);
     try {
       while (true) {
           //  Ожидание клиента
          Socket sc =  listener.accept();
          // Создание в отдельном потоке класса-обработчика
          //  для работы с клиентом (клиенту присваивается
          //  уникальный номер) и старт потока
          new ConverterStr (sc, clientNumber++).start();
        }
     } finally {
         listener.close();
     }
  }
  // Класс-обработчик для преобразования строки
  //    (создается в отдельном потоке).
  // Клиент прерывает соединение при получении пустой строки
  private static class ConverterStr extends Thread {
   private Socket socket;
   private int clientNumber;
   //  Конструктор класса
   public ConverterStr(Socket socket, int clientNumber) {
     this.socket = socket;
     this.clientNumber = clientNumber;
     log("Новое соединение с клиентом#" + clientNumber
            + " в " + socket);
   }
   // Выполняемая работа потока
   public void run() {
     try {
        // Получение входного потока из сокета соединения
        //  с клиентом и преобразование его к
        //  синхронизированному потоку для удобства работы
        BufferedReader in = new BufferedReader(
```

```
            new InputStreamReader(socket.getInputStream()));
          // Получение выходного потока из сокета соединения
          //  с клиентом и преобразование его к другому
          //  потоку для удобства работы со строками
          PrintWriter out = new PrintWriter(
             socket.getOutputStream(), true);
          // Отсылка ответа клиенту об установке соединения
           out.println("Привет, вы являетесь клиентом #"
              + clientNumber + ".");
           out.println(
             "Вводите строки, для завершения пустая строка \n");
           //  Цикл обмена сообщениями с клиентом
           while (true) {
              //  Чтение строки клиента
              String input = in.readLine();
              //  Выход при вводе пустой строки
              if (input == null) {
                   break;
              }
              // Преобразование полученной строки и
              //    передача ее в выходной поток
              out.println(input.toUpperCase());
            }
        } catch (IOException e) {
         log("Ошибка при работе с клиентом#"
               + clientNumber + ": " + e);
        } finally {
          try {
             // Закрытие сокета (соединения) клиента
             socket.close();
          } catch (IOException e) {
            log("Не удалось закрыть сокет");
          }
          log("Соединение с клиентом# "+clientNumber+" закрыто");
        }
    }
    //  Метод логирования
     private void log(String message) {
        System.out.println(message);
     }
   }
}
```

После запуска сервера будет выведено сообщение (рис.8).

```
Search  Debug  Shell  Servers  Console
CapitalizeServer [Java Application] E:\Program Files\Java\jre
Сервер преобразования запущен.
```

Рис.8 Запуск сервера преобразования строк

Класс клиента `ConvStrClient`.

Клиент имеет графический интерфейс, включающий поле для ввода строки и поле многострочного текста для вывода сообщений от сервера. Посылка сообщения при нажатии клавиши Enter после ввода строки. Пустая строка завершает работу клиента.

```
public class ConvStrClient {
   private BufferedReader in;
   private PrintWriter out;
   private JFrame frame = new JFrame("Клиент преобразования строк");
   Socket socket = null;
   // Поле для ввода сообщений
   private JTextField dataField = new JTextField(40);
     //  Область для вывода сообщений от сервера
     private JTextArea messageArea = new JTextArea(8, 60);
     //  Конструктор класса клиента
     public ConvStrClient() {
          // Создание GUI
          messageArea.setEditable(false);
          frame.getContentPane().add(dataField, "North");
          frame.getContentPane().add(
             new JScrollPane(messageArea), "Center");
          // Обработчик события нажатия Enter:
          // Отправка строки на сервер и отображение
          //   полученного результата в поле вывода
          //  При вводе пустой строки - завершение приложения
          dataField.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
               // Запись строки в выходной поток сокета
               out.println(dataField.getText());
               String response;
               try {
                    // Получение ответа
                    response = in.readLine();
                    //  Выход при возврате пустой строки
                    if (response == null||response.equals("")){
                         if (socket!=null)
                            socket.close();
                         System.exit(0);
                    }
               } catch (IOException ex) {
```

```
                    response = "Error: " + ex;
                }
                //  Вывод ответа в области вывода
                messageArea.append(response + "\n");
                dataField.selectAll();
            }
        });
    }
    //  Метод установки соединения с сервером**
    public void connectToServer() throws IOException {
        // Запрос адреса сервера
        String serverAddress = JOptionPane.showInputDialog(
            frame, "Введите IP адрес сервера:",
            "Вход в программу преобразования строк ",
             JOptionPane.QUESTION_MESSAGE);
          // Установка соединения и инициализация потоков
          socket = new Socket(serverAddress, 9898);
          in=new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
          out=new PrintWriter(socket.getOutputStream(),true);
          //Вывод приветствия сервера при установке соединения
          for (int i = 0; i < 3; i++) {
            messageArea.append(in.readLine() + "\n");
          }
    }
    //  Запуск клиентского приложения
    public static void main(String[] args) throws Exception {
      ConvStrClient client = new ConvStrClient();
      client.frame.setDefaultCloseOperation(
           JFrame.EXIT_ON_CLOSE);
      client.frame.pack();
      client.frame.setVisible(true);
      client.connectToServer();
    }

}
```

Затем запускаем клиента. Клиент выводит окно для ввода адреса сервера (рис.9). После ввода адреса (localhost или 127.0.0.1) клиент устанавливает соединение с сервером, получает в ответ приветствие (рис.10). Далее клиент посылает серверу введенную строку и получает в ответ преобразованную строку (рис. 11). Клиент завершает работу при вводе пустой строки.

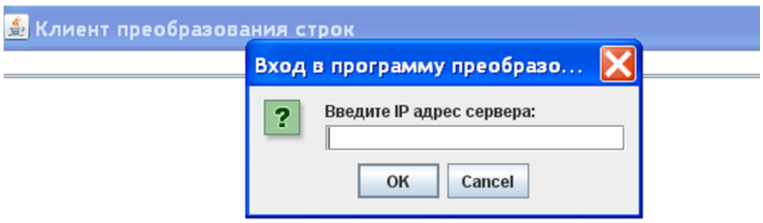

Рис.9 Запуск клиента преобразования строк

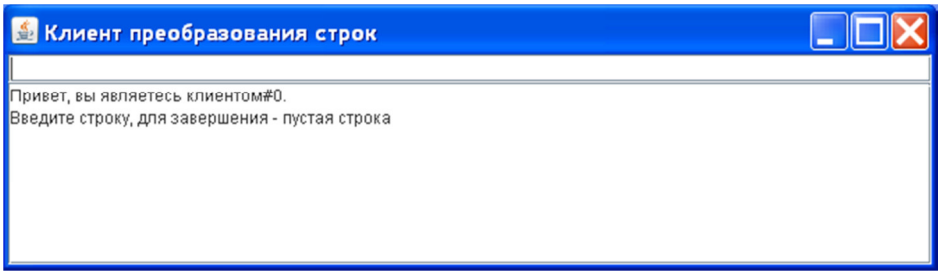

Рис.10 Получение приветствия от сервера

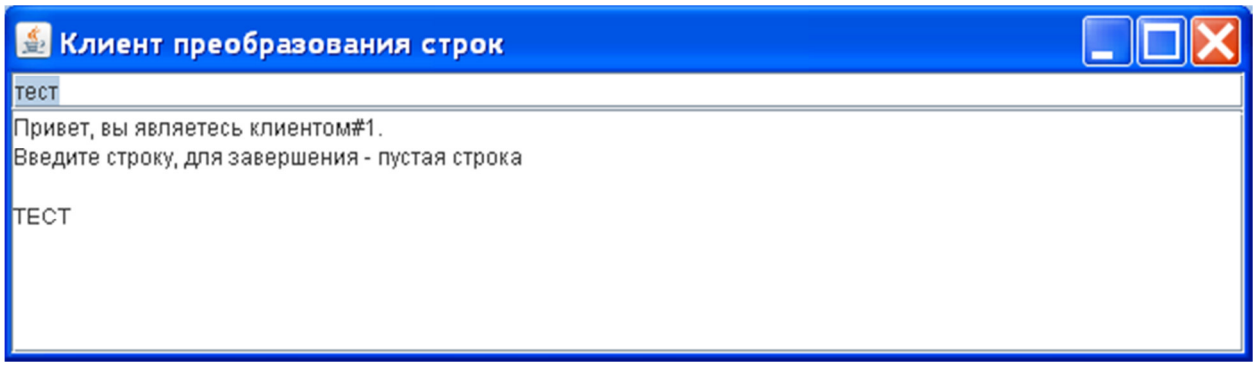

Рис.11 Получение преобразованной строки от сервера

Следующий пример иллюстрирует работу сетевого сервиса, обслуживающего электронную карту клиента.

Упражнение 4. Разработаем систему сетевого сервиса, выполняющего карточное обслуживание клиента. Система будет поддерживать следующий набор операций:

- выдачу клиенту новой карты;
- пополнения счета;
- снятия средств со счета;
- получения остатка средств на карте.

Обмен данными между сервером и клиентом будем выполнять с использованием объектов. От клиента к серверу будем передавать объект операции. От сервера к клиенту будет передаваться объект текстового сообщения о результате ее выполнения операции.

Для обмена объектами по сети задействуем встроенный в Java механизм сериализации (интерфейс `Serializable`).

Данная система реализуется с использованием нескольких классов.

Общими классами для серверного и клиентского приложения являются классы, которыми стороны будут обмениваться по сети.

Для разработки программного приложения системы карточного обслуживания создадим классы:

- класс карты клиента `Card`;
- класс карточной операции `CardOperation`;
- класс сервера `CardServer`;
- серверный класс (сервис) обработки запросов клиента `CardService`;
- класс клиента `CardClient`.

Первый класс `Card` описывает карту клиента. Для сокращения программного кода все поля класса являются общедоступными, а потому не используются методы `get/set`. Объекты класса будут передаваться по сети, поэтому класс должен быть сериализуемым, т.е. поддерживать (реализовывать) интерфейс `Serializable`.

```
public class Card implements Serializable {
  // Поля класса
  public String person;      // Владелец
  public transient Date createDate; // Дата создания
  public String cardNumber;  // Номер карты
  public double balance;     // Остаток средств
  //  Конструктор класса
  public Card(String person, Date createDate,
    String cardNumber,double balance){
    this.person = person;
    this.createDate = createDate;
    this.cardNumber = cardNumber;
    this.balance = balance;
  }
  // Метод вывода информации по карте
  public String toString(){
    return "Карта: Номер="+cardNumber+
           "\tБаланс="+balance+"\tКлиент="+
           person+"\tДата создания="+createDate+"";
  }
}
```

Второй класс описывает операцию с картой. Объект данного класса передается по сети, потому он также должен быть сериализуемым. Одним из полей класса является объект карты. Основными операциями (поле `operTip`) являются: 0 – создание, 1- пополнение, 2-снятие, 3-информация по карте.

```
public class CardOperation implements Serializable {
  //  Поля класса
  public Card card;        // Карта
  public double amount;    // Сумма операции
  public Date operDate;    // Дата операции
  public Integer operTip; // Тип операции
  //  Конструктор класса
  public CardOperation(Card card,Integer operTip,
     double amount,Date operDate){
       this.card = card;
       this.amount = amount;
       this.operDate = operDate;
       this.operTip = operTip;
    }
}
```

Далее рассмотрим классы серверного приложения. Первый из этих классов `CardService` является потоком для обработки запросов, подключившегося клиента. Он также является потомком класса `Thread` и запускается в новом потоке для каждого подсоединенного клиента. Взаимодействие сервера с клиентом выполняется через потоки ввода-вывода. В данном примере используются высокоуровневые потомки потоков ввода и вывода `ObjectInputStream` и `ObjectOutputStream`. Они предоставляют возможность передавать объекты, а не значения примитивных типов.

Передача выполняется просто. Все что нужно сделать – вызвать метод `writeObject` класса `ObjectOutputStream` при передаче и соответственно метод `readObject` класса `ObjectInputStream` при приеме. Поскольку классы `Card` и `CardOperation` объявлены как сериализуемые, экземпляры этих классов при передаче будут автоматически «упакованы», т.е. преобразованы в служебный формат, который может быть сохранен во внешнем файле и отправлен по сети. В этом формате будут сохранены все текущие значения полей объекта, а значит, после восстановления, получится точная копия сохраненного объекта.

```
public class CardService extends Thread {
     ObjectInputStream ins;
     ObjectOutputStream outs;
     CardServer cs;
     Socket sok;
     // Конструктор класса
     public CardService(CardServer cs, Socket sok) {

       this.cs = cs;
       // Извлечение сокета, присланного клиентом
       this.sok = sok;
```

```
    try {
       // Извлечение входного и выходного потоков
      this.outs = new
            ObjectOutputStream(sok.getOutputStream());
      this.ins = new
            ObjectInputStream(sok.getInputStream());
    } catch (IOException e) {
       e.printStackTrace();
    }
    System.out.println("Работают потоки на сокете "
            + sok + "\n");
  }
  //  Операции потока
  public void run() {
    System.out.println(
       "Сервис обслуживания карт стартовал\n");
    boolean work = true;
    String msg;
    Object o;
    //  Цикл обработки запросов клиента
    while (work) {
      try {
        // Чтение присланного объекта из потока сокета
        o = ins.readObject();
        // Проверка типов объекта
        if (o == null) {
          // Пустой объект - выход из цикла
          work = false;
        } else if (o instanceof CardOperation) {
          //  Объект является операцией
          CardOperation co = (CardOperation) o;
          msg = "";
          //  Определение типа операции и ее выполнение
          if (co.operTip == 0) {//Открытие новой карты
            // Выполнение операции
            msg=cs.addNewCard(co.card);
            // Запись операции в выходной поток
            outs.writeObject(msg);
          } else if (co.operTip == 1) {//Зачисление средств
            msg=cs.addMoney(co.card.cardNumber, co.amount);
            outs.writeObject(msg);
          } else if (co.operTip == 2) { // Списание средств
            msg=cs.subMoney(co.card.cardNumber, co.amount);
            outs.writeObject(msg);
          } else if (co.operTip == 3) // Получение остатка
```

```
                outs.writeObject(cs.getCard
                 (co.card.cardNumber));
            } else
                System.out.println("Ошибочная операция");
        } catch (IOException e) {
          e.printStackTrace();
        } catch (ClassNotFoundException e) {
          e.printStackTrace();
        }
      }
    }
}
```

Второй класс серверного приложения – класс сервера `CardServer`. Он является потомком класса `Thread`. Данные о картах хранятся в объекте ассоциированной таблицы: ключ – номер карты, хранимый объект – карта. Класс содержит описание основных операций с картой клиента. Все методы возвращают строку с результатом выполнения операции, которая возвращается клиенту. Сначала сервер создает серверный сокет, который определяется заданным портом. После создания сокет ожидает соединения клиентов. При подключении нового клиента создается экземпляр класса `CardService`, который и занимается всем дальнейшим обслуживанием клиента. Сервис создается в новом потоке, после чего основной поток сервера переключается на прослушивание нового клиента.

```
public class CardServer extends Thread {
     private int srvPort = 7896;
     private ServerSocket ss;
     private Hashtable<String,Card> ht;
    // Конструктор сервера
     public CardServer() {
       // Создание хранилища для карт
       ht = new Hashtable<String,Card> ();
     }
    //  Запуск сервера
     public static void main(String[] args) {
          CardServer cs = new CardServer();
          cs.start();
     }
    //  Запуск потока сервера
     public void run() {
       try {
         // Создание серверного сокета
         ss = new ServerSocket(srvPort);
         log("Сервер стартовал");
         // Цикл обслуживания клиентов
```

```
    while (true) {
      log("Ожидание нового клиента...");
      Socket soc = ss.accept();
      log("Подсоединился клиент");
      // Создание сервиса - потока обслуживания клиента
      CardService service = new CardService(this, soc);
      log("Создан сервис обслуживания карт");
      service.start();
    }
  }catch (IOException e) {
    e.printStackTrace();
  }
}
// Метод создания новой карты
public String addNewCard(Card card) {
  Card c = (Card) ht.get(card.cardNumber);
  String msg;
  // Проверка наличия карты с заданным номером
  if (c != null) {
    msg= "Ошибка. Карта с таким номером существует\n";
    log(msg);
    return msg;
  }
  else {
    // Запись карты в хранилище
    ht.put(card.cardNumber, card);
    msg="Создана карта с номером"+card.cardNumber;
    log(msg);
    return msg;
  }
}
//  Метод пополнения карты
public String addMoney(String card, double money) {
  // Извлечение карты из хранилища
  Card c = (Card) ht.get(card);
  String msg;
  // Проверка наличия заданной карты
  if (c == null) {
    msg= "Ошибка. Неправильный номер карты\n";
    log(msg);
    return msg;
  };
  // Зачисление суммы
  c.balance += money;
  // Запись карты в хранилище
```

```
    ht.put(card, c);
    msg="Пополнение счета на "+money+" с карты# "
      +card+". Остаток=" + c.balance+"\n";
    log(msg);
    return msg;
  }
  // Снятие со счета
  public String subMoney(String card, double money) {
    // Извлечение карты из хранилища
    Card c = (Card) ht.get(card);
    String msg;
    // Проверка наличия заданной карты
    if (c == null) {
      // Проверка наличия заданной карты
      msg= "Ошибка. Неправильный номер карты\n";
      log(msg);
      return msg;
    };
    // Проверка баланса
    if (c.balance < money){
      msg="Ошибка. Недостаточно средств. На карте только "
        + c.balance+"\n";
      log(msg);
      return msg;
    }
    else {
      // Снятие суммы
      c.balance -= money;
      // Запись карты в хранилище
      ht.put(card, c);
      msg="Снятие суммы "+money+" с карты# "+card+
         ". Остаток=" + c.balance+"\n";
      log(msg);
      return msg;
    }
  }
  // Выдача информации по карте
  public String getCard(String card) {
    // Извлечение карты из хранилища
    Card c = (Card) ht.get(card);
    if (c == null) {
      // Проверка наличия заданной карты
      String msg= "Ошибка. Неправильный номер карты\n";
      log(msg);
      return msg;
```

```
        };
        log("Запрос остатка средств по карте#"+card);
        return c.toString();
    }
    // Метод логирования (вывод сообщений сервера в консоль)
    private void log (String msg) {
        System.out.println(msg);
    }
}
```

После запуска сервера в консоль будет выведено сообщение о готовности сервера к приему сообщений от клиента (рис. 12).

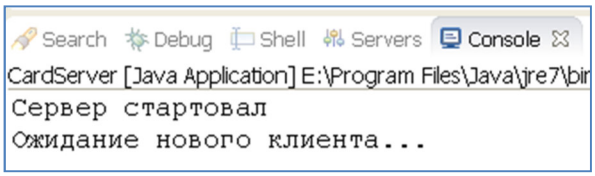

Рис.12 Запуск сервера

В завершение создадим класс клиента `CardClient`, который также как и серверный процесс использует для передачи высокоуровневые потоки ввода и вывода `ObjectInputStream` и `ObjectOutputStream`, извлеченные из сокета. Класс соединяется с сервером и посылает ему набор тестовых операций с картой.

```
public class CardClient {
    int tipOper;
    int srvPort = 7896;
    String srvName;
    Socket s;
    ObjectInputStream ins;
    ObjectOutputStream outs;
    // Конструктор
    public CardClient(String serverName) {
        this.srvName = serverName;
    }
    // Метод тестирования операций
    public void startTest() throws
        IOException, ClassNotFoundException {
        connectToServer();
        CardOperation co = new CardOperation(
            new Card("Иван", new Date(), "1",
            0.0), 0, 0, new Date());
        processOperation(co);
        co = new CardOperation(new Card("Алексей",
            new Date(),"2",0.0),0,0,new Date());
```

```
        processOperation(co);
        co = new CardOperation(new Card("Ольга",
             new Date(),"3",0.0),0,0,new Date());
        processOperation(co);
        co = new CardOperation(new Card("Nataly",
             new Date(),"3",0.0),0,0,new Date());
        processOperation(co);
        co = new CardOperation(new Card(null, null, "1", 0),
              1, 1200, new Date());
        processOperation(co);
        co = new CardOperation(new Card(null, null, "2", 0),
              1, 300, new Date());
        processOperation(co);
        co = new CardOperation(new Card(null, null, "3", 0),
              1, 500, new Date());
        processOperation(co);
        co = new CardOperation(new Card(null, null, "3", 0),
              2, 1000, new Date());
        processOperation(co);
        co = new CardOperation(new Card(null, null, "3", 0),
              2, 200, new Date());
        processOperation(co);
        co = new CardOperation(new Card(null, null, "1", 0),
              3, 0, new Date());
        processOperation(co);
        co = new CardOperation(new Card(null, null, "2", 0),
             3, 0, new Date());
        processOperation(co);
        co = new CardOperation(new Card(null, null, "3", 0),
              3, 0, new Date());
        processOperation(co);
        outs.writeObject(null);
        s.close();
        log("Работа завершена");
    }
    // Соединение с сервером
    void connectToServer()
      throws UnknownHostException, IOException {
        s = new Socket(srvName, srvPort);
        log("Соединение с сервером установлено\n");
        ins = new ObjectInputStream(s.getInputStream());
        outs = new ObjectOutputStream(s.getOutputStream());
        log("Созданы потоки для обмена данными\n");
    }
    // Метод выполнения операции
```

```
    void processOperation(CardOperation co)
        throws IOException,ClassNotFoundException {
        log("Операция " + co.operTip + " с картой "
                + co.card.cardNumber);
        //  Отсылка операции серверу
        outs.writeObject(co);
        //  Вывод ответа сервера
        log("Результат: "+ (String) ins.readObject());
    }
    // Запуск клиента
    public static void main(String[] args) throws Exception {
        String serverAddress=JOptionPane.showInputDialog(
            null,"Введите IP адрес сервера:",
            "Вход в программу операций по картам",
            JOptionPane.PLAIN_MESSAGE);
        CardClient bc = new CardClient(serverAddress);
        try {
            bc.startTest();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private void log (String msg) {
        System.out.println(msg);
    }

}
```

При запуске клиента запрашивается адрес сервера (как в предыдущем упражнении), в качестве которого введем localhost.

После запуска клиента в консоли клиента выводится информация о выполненных операциях (рис.13). В консоли сервера также выполняется логирование выполненных операций (рис. 14).

```
Search  Debug  Shell  Servers  Console
<terminated> CardClient [Java Application] E:\Program Files\Java\jre7\bin\javaw.exe (08 июля 2017 г., 20:29:26)
Созданы потоки для обмена данными

Операция 0 с картой 1
Результат: Создана карта с номером1
Операция 0 с картой 2
Результат: Создана карта с номером2
Операция 0 с картой 3
Результат: Создана карта с номером3
Операция 0 с картой 3
Результат: Ошибка. Карта с таким номером существует

Операция 1 с картой 1
Результат: Пополнение счета на 1200.0 с карты# 1. Остаток=1200.0

Операция 1 с картой 2
Результат: Пополнение счета на 300.0 с карты# 2. Остаток=300.0

Операция 1 с картой 3
Результат: Пополнение счета на 500.0 с карты# 3. Остаток=500.0

Операция 2 с картой 3
Результат: Ошибка. Недостаточно средств. На карте только 500.0

Операция 2 с картой 3
Результат: Снятие суммы 200.0 с карты# 3. Остаток=300.0

Операция 3 с картой 1
Результат: Карта: Номер=1       Баланс=1200.0   Клиент=Иван     Дата создания=Sat
Операция 3 с картой 2
Результат: Карта: Номер=2       Баланс=300.0    Клиент=Алексей  Дата создания=Sat
Операция 3 с картой 3
Результат: Карта: Номер=3       Баланс=300.0    Клиент=Ольга    Дата создания=Sat
Работа завершена
```

Рис.13 Работа клиентского приложения

```
Search  Debug  Shell  Servers  Console
CardServer [Java Application] E:\Program Files\Java\jre7\bin\javaw.exe (08 июля 2017 г., 20:17:46)
Сервер стартовал
Ожидание нового клиента...
Подсоединился клиент
Работают потоки на сокете Socket[addr=/127.0.0.1,port=1070,localport=7896]

Создан сервис обслуживания карт
Ожидание нового клиента...
Сервис обслуживания карт стартовал

Создана карта с номером1
Создана карта с номером2
Создана карта с номером3
Ошибка. Карта с таким номером существует

Пополнение счета на 1200.0 с карты# 1. Остаток=1200.0

Пополнение счета на 300.0 с карты# 2. Остаток=300.0

Пополнение счета на 500.0 с карты# 3. Остаток=500.0

Ошибка. Недостаточно средств. На карте только 500.0

Снятие суммы 200.0 с карты# 3. Остаток=300.0

Запрос остатка средств по карте#1
Запрос остатка средств по карте#2
Запрос остатка средств по карте#3
```

Рис.14 Протокол работы сервера

2. Задание

1. Разработать клиент-серверное приложение с использованием протокола UDP

2. Разработать клиент-серверное приложение с использованием протокола TCP и сокетов